Lecture 11 Relational Database Design
3rd Normal Form

United International College

Outline

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Functional Dependency Theory
- BCNF
- 3rd Normal Form
- Multivalued Dependencies and Decomposition
- Database-Design Process

Motivation

- There are some situations that
  - BCNF decomposition is not dependency preserving, but
  - efficiently checking the dependency violation on updates is important.
- Thus, we define a weaker normal form called Third Normal Form (3NF).
  - 3NF allows some redundancy (with resultant problems; we will see examples later).
  - But functional dependencies can be checked on individual relations without computing a join.
  - There is always a lossless-join, dependency-preserving decomposition into 3NF.

Third Normal Form

- A relation schema R is in 3NF with respect to a set F of functional dependencies if for all functional dependencies in F^+ of the form

$$\alpha \rightarrow \beta$$

  at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial ($\beta \subseteq \alpha$);
  - α is a superkey for R; or
  - each attribute in $\beta \setminus \alpha$ is contained in a candidate key for R.
    (**NOTE***:* each attribute can be in a different candidate key)
- If a relation is in BCNF it is in 3NF (since in BCNF one of the first two conditions above must hold).
- Third condition is a minimal relaxation of BCNF to ensure dependency preservation (will see why later).

3NF Example

- Recall the example which shows BCNF is not dependency preserving. Let's try it in 3NF.

  $$R = \{J, K, L\}$$

  $$F = \{JK \rightarrow L, L \rightarrow K\}$$

  Candidate keys are $\{J, K\}$ and $\{J, L\}$.

- R is already in 3NF because
  - $JK \rightarrow L$ satisfies condition 2. $\{J, K\}$ is a superkey.
  - $L \rightarrow K$ satisfies condition 3. $\{K\} \setminus \{L\} = \{K\}$ is a subset of the candidate key $\{J, K\}$.

Redundancy in 3NF

- We allow redundancies in 3NF.
- For example, $R = \{J, K, L\}$, $F = \{JK \rightarrow L, L \rightarrow K\}$

J	*L*	*K*
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

3NF Decomposition Strategy

- In order to make sure the decomposition is dependency preserving, we decompose the schema by using every functional dependency in an **equivalent** functional dependency set.
- To keep the decomposition simple, we find the "**smallest**" FD set (canonical cover)
- by removing all "**useless**" (extraneous) attributes from each of the FD.
  - We have seen "equivalent" FD set, the closures.
  - If two FD sets are equivalent, then one can logically infer the other.
  - In other words, the constraints presented by one schema is exactly the same as the other.

Extraneous Attributes

- Intuitively, the extraneous attributes are the attributes which can be removed without changing the constraints expressed by the functional dependency set.
- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.
  - Attribute $A \in \alpha$ is **extraneous** if A and F logically implies the new functional dependency set $F' = (F \setminus \{\alpha \rightarrow \beta\}) \cup \{(\alpha \setminus \{A\}) \rightarrow \beta\}$.
  - Attribute $A \in \beta$ is **extraneous** if A and the new set of functional dependencies $F' = (F \setminus \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta \setminus \{A\})\}$ logically implies F.
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$ (I.e. the result of dropping B from $AB \rightarrow C$).
- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
  - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C.

Extraneous Attributes Detection

- Consider a set of functional dependencies F and the functional dependency $\alpha \rightarrow \beta$ in F.
- To test if attribute $A \in \alpha$ is extraneous in α,
  1. compute $(\alpha \setminus \{A\})^+$ using the dependencies in F, and
  2. check that whether $(\alpha \setminus \{A\})^+$ contains β; if it does, A is extraneous in α.
- To test if attribute $A \in \beta$ is extraneous in β,
  1. compute α^+ using only the dependencies in
     $$F' = (F \setminus \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta \setminus \{A\})\}, \text{ and}$$
  2. check that whether α^+ contains A; if it does, A is extraneous in β.

Canonical Cover

- Intuitively, a **canonical cover** of a set of dependencies F is the "minimal" set of FDs equivalent to F.
- No redundant dependencies or extraneous attributes in any one of the dependencies.
- Formally, A **canonical cover** for F is a set of dependencies F_c such that
  - F logically implies all dependencies in F_c, and
  - F_c logically implies all dependencies in F, and
  - No functional dependency in F_c contains an extraneous attribute, and
  - Each left side of functional dependency in F_c is unique.
- For example,
  - $\{A \rightarrow C, B \rightarrow C, A \rightarrow C\}$
  - $\{A \rightarrow C, B \rightarrow C, A \rightarrow CD\}$
  - $\{A \rightarrow C, B \rightarrow C, AC \rightarrow D\}$

Find Canonical Cover

Algorithm Compute the canonical cover F_c for F

1. $F_c = F$
2. **repeat**
3. **if** $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ are in F_c
4. Combine them as $\alpha_1 \rightarrow \beta_1\beta_2$ using the union rule
5. **end if**
6. **For each** FD $\alpha \rightarrow \beta \in F_c$
7. remove extraneous attributes if there is any
8. **end for**
9. **until** F_c does not change
10. **return** F_c

- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied

Example

- $R = \{A, B, C\}, F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
- Step 1, combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$

  And, $F_c = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- A is extraneous in $AB \rightarrow C$.
  - Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies.
  - Yes: in fact, $B \rightarrow C$ is already presented.
  - Then, $F_c = \{A \rightarrow BC, B \rightarrow C\}$
- C is extraneous in $A \rightarrow BC$.
  - Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies.
  - Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
  - Can use attribute closure of A in more complex cases.
- The canonical cover is: $F_c = \{A \rightarrow B, B \rightarrow C\}$.

More Examples

$$R = \{A, B, C, D, E, F, G, H\}$$
$$F = \{AC \rightarrow G, D \rightarrow EG, BC \rightarrow D, CG \rightarrow BD, ACD \rightarrow B, CE \rightarrow AG\}$$

First iteration:

- Union
- Find a redundant (extraneous) attribute

  Consider $ACD \rightarrow B$, D is extraneous, because $AC \rightarrow G$ and $CG \rightarrow BD$ imply $AC \rightarrow CG \rightarrow B$.
- So, replace $ACD \rightarrow B$ by $AC \rightarrow B$ and result

  $F_c = \{AC \rightarrow G, D \rightarrow EG, BC \rightarrow D, CG \rightarrow BD, AC \rightarrow B, CE \rightarrow AG\}$

More Examples

$$R = \{A, B, C, D, E, F, G, H\}$$
$$F_c = \{AC \rightarrow G, D \rightarrow EG, BC \rightarrow D, CG \rightarrow BD, AC \rightarrow B, CE \rightarrow AG\}$$

Second iteration:

- Union
  $$\{AC \rightarrow BG, D \rightarrow EG, BC \rightarrow D, CG \rightarrow BD, CE \rightarrow AG\}$$
- Find extraneous attributes

  Consider $AC \rightarrow BG$, G is extraneous, because $AC \rightarrow B$, $BC \rightarrow D$, and $D \rightarrow EG$ imply $AC \rightarrow D \rightarrow G$.
- So, replace $AC \rightarrow BG$ by $AC \rightarrow B$ and result

  $$F_c = \{AC \rightarrow B, D \rightarrow EG, BC \rightarrow D, CG \rightarrow BD, CE \rightarrow AG\}$$

More Examples

$$R = \{A, B, C, D, E, F, G, H\}$$
$$F_c = \{AC \rightarrow B, D \rightarrow EG, BC \rightarrow D, CG \rightarrow BD, CE \rightarrow AG\}$$

Third iteration:

- Union
- Find extraneous attributes

  Consider $CG \rightarrow BD$, B is extraneous, because $CG \rightarrow D$, $D \rightarrow EG$, $CE \rightarrow AG$, and $AC \rightarrow B$ imply $CG \rightarrow CD \rightarrow CE \rightarrow AC \rightarrow B$.
- So, replace $CG \rightarrow BD$ by $CG \rightarrow D$ and result

  $F_c = \{AC \rightarrow B, D \rightarrow EG, BC \rightarrow D, CG \rightarrow D, CE \rightarrow AG\}$

More Examples

$$R = \{A, B, C, D, E, F, G, H\}$$
$$F_c = \{AC \rightarrow B, D \rightarrow EG, BC \rightarrow D, CG \rightarrow D, CE \rightarrow AG\}$$

Fourth iteration:

- Union
- Find extraneous attributes

  Consider $CE \rightarrow AG$, G is extraneous, because $CE \rightarrow A$, $AC \rightarrow B$, $BC \rightarrow D$, and $D \rightarrow EG$ imply $CE \rightarrow AC \rightarrow BC \rightarrow D \rightarrow G$.
- So, replace $CE \rightarrow AG$ by $CE \rightarrow A$ and result

  $F_c = \{AC \rightarrow B, D \rightarrow EG, BC \rightarrow D, CG \rightarrow D, CE \rightarrow A\}$

More Examples

$$R = \{A, B, C, D, E, F, G, H\}$$
$$F_c = \{AC \rightarrow B, D \rightarrow EG, BC \rightarrow D, CG \rightarrow D, CE \rightarrow A\}$$

- No more extraneous attributes. F_c is the canonical cover.

Ununiqueness of the Canonical Cover

- Different order in which the extraneous attributes are considered can result in different F_c.
- For example,

$$R = \{A, B, C\},\ F = \{A \rightarrow BC, B \rightarrow AC, C \rightarrow AB\}$$

- For $A \rightarrow BC$, we find that both B and C are extraneous under F.
- However, we cannot remove both.
- In the end, all the following are valid canonical covers:

$$F_c = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$$
$$F_c = \{A \rightarrow B, B \rightarrow AC, C \rightarrow B\}$$
$$F_c = \{A \rightarrow C, C \rightarrow B, B \rightarrow A\}$$
$$F_c = \{A \rightarrow C, B \rightarrow C, C \rightarrow AB\}$$

3NF Decomposition

Algorithm 3NF Decomposition

1. Let F_c be a canonical cover for F
2. $i = 0$
3. **for each** functional dependency $\alpha \rightarrow \beta$ in F_c **do**
4. **if** none of the schemas R_j, $1 \leq j \leq i$ contains $\alpha \cup \beta$ **then**
5. $i = i + 1$
6. $R_i = \alpha \cup \beta$
7. **end if**
8. **end for**
9. **if** none of the schemas R_j, $1 \leq j \leq i$ contains a candidate key for R **then**
10. $i = i + 1$
11. R_i= any candidate key for R
12. **end if**
13. **return** $\{R_1, R_2, \ldots, R_i\}$

3NF Decomposition Example

- Relation schema:
  $cust\_banker\_branch = \{\underline{customer\_id, employee\_id}, branch\_name, type\}$
- The functional dependencies for this relation schema are:
  1. $type \rightarrow customer\_id$
  2. $customer\_id, employee\_id \rightarrow branch\_name, type$
  3. $employee\_id \rightarrow branch\_name$
- **First step:** compute the canonical cover
  - $branch\_name$ is extraneous in the right-hand side of the 2nd dependency.
  - No other attributes are extraneous, so we get
  - $F_c = \{\ type \rightarrow customer\_id,$
    $customer\_id, employee\_id \rightarrow type,$
    $employee\_id \rightarrow branch\_name\}$
- **Note:** this is only an example to show the decomposition.
  $type \rightarrow customer\_id$ cannot be true in real cases.

3NF Decomposition Example

- Relation schema:
  $cust\_banker\_branch = \{\underline{customer\_id, employee\_id}, branch\_name, type\}$
- The canonical cover $F_c =$
  1. $type \rightarrow customer\_id$
  2. $customer\_id, employee\_id \rightarrow type$
  3. $employee\_id \rightarrow branch\_name$
- **Second step:** decompose the schema using every FD in F_c
  - $R_1 = \{\underline{type}, customer\_id\}$
  - $R_2 = \{\underline{customer\_id, employee\_id}, type\}$
  - $R_3 = \{\underline{employee\_id}, branch\_name\}$
- R_2 contains a candidate key of the original schema, so no further relation schema is required.

3NF Decomposition Example

- Relation schema:
  $cust\_banker\_branch = \{\underline{customer\_id, employee\_id}, branch\_name, type\}$
- The canonical cover $F_c =$
  1. $type \rightarrow customer\_id$
  2. $customer\_id, employee\_id \rightarrow type$
  3. $employee\_id \rightarrow branch\_name$
- Considering FDs in different order may result in different decompositions.
- If we consider FD2 before FD1, the result will be
  - $R'_1 = \{\underline{customer\_id, employee\_id}, type\}$
  - $R'_2 = \{\underline{employee\_id}, branch\_name\}$

  because $\{\underline{type}, customer\_id\}$ is a subset of $\{\underline{customer\_id, employee\_id}, type\}$.

3NF Decomposition Example

- Minor extension of the 3NF decomposition algorithm: at end of the for loop, detect and delete schemas, such as $\{\underline{type}, customer\_id\}$, which are subsets of other schemas.
- Then, the result will not depend on the order in which FDs are considered.
- The resultant simplified 3NF schema is:

$$R_1 = \{\underline{customer\_id, employee\_id}, type\}$$

$$R_2 = \{\underline{employee\_id}, branch\_name\}$$

Comparison of BCNF and 3NF

- It is always possible to decompose a relation into a set of relations that are in 3NF such that:
  - the decomposition is lossless.
  - the dependencies are preserved.
- It is always possible to decompose a relation into a set of relations that are in BCNF such that:
  - the decomposition is lossless.
  - it may not be possible to preserve dependencies.

End of Lecture 11